



Université Mohammed V
Faculté des Sciences Rabat

Département Informatique
Master Informatique Gouvernance et Transformation
Digitale

Projet Fin de Semestre

Titre :

Système de Surveillance Active de la
Pollution de l'Air avec Alerte Intelligente
par Email

Réalisé par :

Nirmine Hiani

Hajar Chahbi

Encadré par :

Mme Sara Diouani

Rabat - Année universitaire 2025-2026

Table des matières

1	Introduction générale et contexte du projet	4
1.1	Contexte général	4
1.2	Problématique	5
1.3	Objectifs du projet	5
2	Présentation des technologies et outils utilisés	6
2.1	Introduction	6
2.2	Architecture générale de la solution	6
2.3	Microsoft Azure	7
2.4	Supabase	7
2.5	API de données météorologiques et de pollution	7
2.6	EmailJS	8
2.7	Outils de développement	8
3	Déploiement de la solution sur Microsoft Azure	9
3.1	Objectif du chapitre	9
3.2	Choix de la plateforme cloud : Microsoft Azure	9
3.3	Architecture générale de la solution	10
3.4	Mise en place du Storage Account	11
3.5	Déploiement de l'application web	12
3.6	Sécurité et accessibilité	13
4	Présentation et fonctionnement de l'application Pollution Dashboard	14
4.1	Objectif de l'application	14
4.2	Accès à l'application	14
4.3	Architecture générale de l'application	15
4.4	Tableau de suivi de la qualité de l'air	15
4.5	Visualisation graphique des données	16

4.6	Carte interactive	16
4.7	Fonctionnalité de recherche	17
4.8	Synthèse du chapitre	17
4.9	Limites fonctionnelles de l'application	18
5	Présentation de la plateforme Supabase	19
5.1	Supabase	19
5.1.1	Architecture de Supabase	19
5.1.2	Base de données et création des tables	20
5.1.3	API automatique	21
5.1.4	Edge Functions	21
5.1.5	Authentification et sécurité	25
5.2	Justification du choix des technologies	25
6	Fonctionnement global de l'application Pollution Predictor avec supabase et alerte par Gmail	27
6.1	Présentation générale de l'application	27
6.2	Fonctionnement du dashboard de qualité de l'air	28
6.2.1	Interaction utilisateur et récupération des données	28
6.2.2	Indicateurs de pollution affichés	28
6.2.3	Historique et visualisation graphique	29
6.3	Inscription aux alertes de pollution	29
6.3.1	Formulaire d'inscription	29
6.3.2	Stockage des inscriptions	30
6.4	Module Contact et Désinscription	30
6.4.1	Formulaire de contact	30
6.4.2	Gestion côté backend	31
6.5	Système d'alertes email	32
6.5.1	Configuration du service email	32
6.5.2	Template d'email	32
6.5.3	Réception des alertes	33
7	Conclusion générale	34

Table des figures

3.1	Choix de la plateforme Microsoft Azure pour l'hébergement du projet.	10
3.2	Architecture générale de la solution (frontend statique + Azure Storage + APIs).	11
3.3	Conteneur <code>\$web</code> dans Azure Storage Explorer pour l'hébergement du site statique.	12
3.4	Activation de l'hébergement statique et génération de l'URL publique (endpoint).	13
4.1	Tableau de suivi de la qualité de l'air	15
4.2	Graphiques d'évolution de la qualité de l'air	16
4.3	Carte interactive de la qualité de l'air	17
5.1	Création d'une table dans Supabase	20
5.2	RLS désactivé dans Supabase	21
5.3	Gestion des Edge Functions dans Supabase	22
5.4	Vue d'ensemble d'une Edge Function	23
5.5	Historique des invocations d'une Edge Function	23
5.6	Logs d'exécution d'une Edge Function	24
5.7	Code source d'une Edge Function	24
5.8	Section <i>Details</i> d'une Edge Function Supabase	25
6.1	Dashboard de qualité de l'air avec carte interactive	28
6.2	Historique de la qualité de l'air (AQI et PM2.5)	29
6.3	Formulaire d'inscription aux alertes de pollution	30
6.4	Table des utilisateurs inscrits aux alertes	30
6.5	Formulaire de contact et de désinscription	31
6.6	Table des messages et désinscriptions	31
6.7	Configuration du service email avec EmailJS	32
6.8	Template d'email pour les alertes de pollution	32
6.9	Exemple d'alerte de pollution reçue par email	33

Chapitre 1

Introduction générale et contexte du projet

1.1 Contexte général

Avec l'évolution rapide des technologies du *cloud computing* et la généralisation des applications web, la gestion et l'exploitation des données en temps réel sont devenues des éléments essentiels dans de nombreux domaines, notamment celui de l'environnement. Parmi les problématiques les plus importantes aujourd'hui figure la pollution atmosphérique, qui impacte directement la santé des populations et la qualité de vie dans les grandes villes.

Grâce à l'ouverture des données publiques et à la disponibilité d'API météorologiques et environnementales, il est désormais possible d'accéder à des informations sur la qualité de l'air, les conditions climatiques et les niveaux de pollution dans différentes régions. Toutefois, ces données restent souvent dispersées, peu exploitées ou difficilement accessibles pour les utilisateurs finaux.

Dans ce contexte, le *cloud computing* offre une solution efficace permettant de centraliser les données, de les traiter, de les stocker et de les visualiser à travers des interfaces modernes accessibles à distance.

C'est dans cette optique que s'inscrit ce projet, qui consiste à concevoir une application cloud de surveillance de la pollution, basée sur l'exploitation d'API externes et de services cloud tels que Microsoft Azure et Supabase.

Ce projet s'inscrit dans une démarche de sensibilisation à la qualité de l'air et à l'utilisation des technologies cloud pour la collecte et l'exploitation de données environnementales accessibles au grand public.

1.2 Problématique

Malgré la disponibilité de nombreuses données environnementales en ligne, plusieurs limites persistent :

- les données sont souvent dispersées sur différentes plateformes ;
- l'accès aux informations n'est pas toujours intuitif pour les utilisateurs ;
- l'analyse et la visualisation des niveaux de pollution restent limitées ;
- peu d'outils permettent une centralisation simple et efficace des données.

De plus, les citoyens et les décideurs manquent souvent d'outils clairs permettant de consulter l'état de la pollution dans une ville donnée et d'en suivre l'évolution.

Ainsi, la problématique principale de ce projet peut être formulée comme suit :

Comment concevoir une application cloud permettant de collecter, stocker, analyser et visualiser les données de pollution à partir d'API externes, de manière simple, fiable et accessible ?

1.3 Objectifs du projet

L'objectif principal de ce projet est de développer une solution cloud permettant de surveiller la pollution atmosphérique à partir de données en ligne, tout en exploitant les services modernes du cloud computing.

Les objectifs spécifiques sont les suivants :

- collecter les données de pollution à partir d'une API météo ;
- centraliser les données dans une base cloud sécurisée ;
- exploiter des services cloud pour le traitement et la gestion des données ;
- développer une interface permettant de visualiser les niveaux de pollution par ville ;
- mettre en place une architecture simple, *scalable* et réutilisable ;
- mettre en pratique les notions de *cloud computing*.

Chapitre 2

Présentation des technologies et outils utilisés

2.1 Introduction

Ce chapitre présente les différentes technologies et outils utilisés pour la réalisation du projet. L'objectif est de décrire l'environnement technique choisi, ainsi que le rôle de chaque composant dans la mise en place de la solution cloud de surveillance de la pollution.

Le projet repose principalement sur une architecture cloud combinant l'utilisation de services modernes tels que Microsoft Azure, Supabase ainsi qu'une API externe permettant l'accès aux données de pollution atmosphérique.

2.2 Architecture générale de la solution

La solution développée repose sur une architecture cloud orientée services. Elle permet :

- la récupération des données de pollution via une API externe ;
- le stockage et la gestion des données dans une base cloud ;
- le traitement et l'exploitation des données ;
- l'affichage des résultats sur une interface web.

Cette architecture permet une séparation claire entre la couche de données, la couche de traitement et la couche de présentation, ce qui améliore la maintenabilité et l'évolutivité du système.

2.3 Microsoft Azure

Microsoft Azure est une plateforme de cloud computing proposée par Microsoft. Elle offre un ensemble de services permettant de développer, héberger et gérer des applications via des centres de données sécurisés.

Dans le cadre de ce projet, Azure est utilisé pour :

- l'hébergement de l'application ;
- la gestion des services cloud ;
- l'exécution des traitements liés aux données ;
- l'assurance de la disponibilité et de la scalabilité du système.

Azure a été choisi pour sa fiabilité, sa flexibilité et son intégration facile avec des services web modernes.

2.4 Supabase

Supabase vous permet de gérer vos données, d'authentifier les utilisateurs, de stocker des fichiers et même d'exécuter des fonctions serverless sans vous soucier de configurations complexes.

Dans le cadre de ce projet, Supabase est utilisé pour :

- la gestion de la base de données PostgreSQL, incluant le stockage des utilisateurs, des données de pollution et des messages de contact ;
- la mise à disposition d'une API automatique permettant d'interagir avec les tables depuis le frontend ;
- l'hébergement et l'exécution des *Edge Functions*
- la sécurisation des accès et la gestion des règles de sécurité
- la surveillance et l'historisation des données de pollution, permettant la génération de graphiques et d'alertes.

Supabase a été choisi pour les raisons suivantes :

- sa simplicité d'intégration avec des projets basés sur JavaScript et TypeScript ;
- sa flexibilité, permettant de combiner frontend, backend et base de données au sein d'un même environnement ;

2.5 API de données météorologiques et de pollution

Les données de pollution sont récupérées à partir d'une API externe fournissant des informations telles que :

- la qualité de l'air ;
- les indices de pollution ;
- les conditions météorologiques ;
- la localisation par ville.

L'utilisation d'une API permet d'obtenir des données mises à jour en temps réel, sans avoir besoin de capteurs physiques, ce qui rend la solution plus simple à déployer et plus économique.

2.6 EmailJS

EmailJS est un service cloud permettant l'envoi d'e-mails directement depuis une application web, sans nécessiter la mise en place d'un serveur backend dédié. Il a été intégré dans ce projet afin d'assurer la gestion des notifications automatiques liées à la qualité de l'air.

Dans le cadre de cette application, EmailJS est utilisé pour :

- envoyer des notifications par e-mail aux utilisateurs ;
- alerter en cas de dépassement des seuils de pollution ;
- renforcer l'aspect préventif et informatif de la plateforme.

EmailJS s'intègre naturellement dans l'architecture cloud de l'application et complète les fonctionnalités existantes. L'intégration de ce service repose sur l'utilisation de modèles d'e-mails prédéfinis et d'une clé d'authentification sécurisée fournie par EmailJS. Cette approche permet d'éviter la mise en place d'un serveur de messagerie côté backend, tout en garantissant une communication fiable et rapide avec les utilisateurs.

2.7 Outils de développement

Pour le développement du projet, plusieurs outils ont été utilisés :

- un langage de programmation web pour l'interface utilisateur ;
- des bibliothèques JavaScript pour l'affichage dynamique ;
- un éditeur de code (Visual Studio Code) ;
- des outils de test et de débogage intégrés au navigateur.

Ces outils ont permis de concevoir une application fonctionnelle, interactive et facile à maintenir.

Chapitre 3

Déploiement de la solution sur Microsoft Azure

3.1 Objectif du chapitre

Ce chapitre présente la mise en place de l'infrastructure cloud utilisée pour héberger l'application de surveillance de la qualité de l'air. Il se concentre exclusivement sur l'environnement technique, le déploiement et les choix d'architecture cloud, sans aborder le fonctionnement fonctionnel de l'application ni l'analyse des données, qui seront détaillés dans les chapitres suivants.

L'objectif principal est de montrer comment l'application a été rendue accessible en ligne grâce à Microsoft Azure, tout en garantissant simplicité, performance et disponibilité.

3.2 Choix de la plateforme cloud : Microsoft Azure

Microsoft Azure a été choisi comme plateforme cloud pour les raisons suivantes :

- plateforme fiable et largement utilisée dans le domaine professionnel ;
- prise en charge native des applications web statiques ;
- disponibilité d'une offre gratuite adaptée aux projets académiques ;
- simplicité de déploiement sans gestion de serveur ;
- compatibilité avec les technologies front-end (HTML, CSS, JavaScript).

Azure permet d'adopter une approche **PaaS (Platform as a Service)**, ce qui signifie que l'utilisateur ne gère ni l'infrastructure matérielle ni le système d'exploitation.

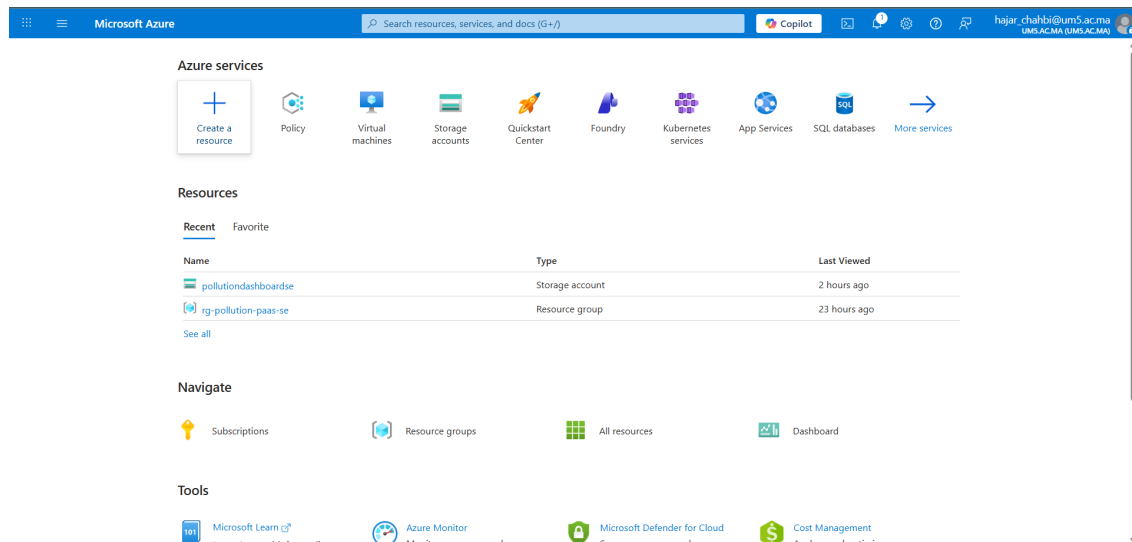


FIGURE 3.1 – Choix de la plateforme Microsoft Azure pour l'hébergement du projet.

3.3 Architecture générale de la solution

L'architecture retenue repose sur une structure simple et efficace :

- **Frontend** : application web statique (HTML, CSS, JavaScript)
- **Hébergement** : Azure Storage – Static Website
- **Accès utilisateur** : navigateur web via une URL publique
- **Données** : récupérées dynamiquement depuis des API externes
- **Cartographie** : Leaflet avec OpenStreetMap

Cette architecture permet :

- une exécution rapide côté client,
- une maintenance simplifiée,
- une bonne évolutivité,
- un coût d'hébergement très faible.

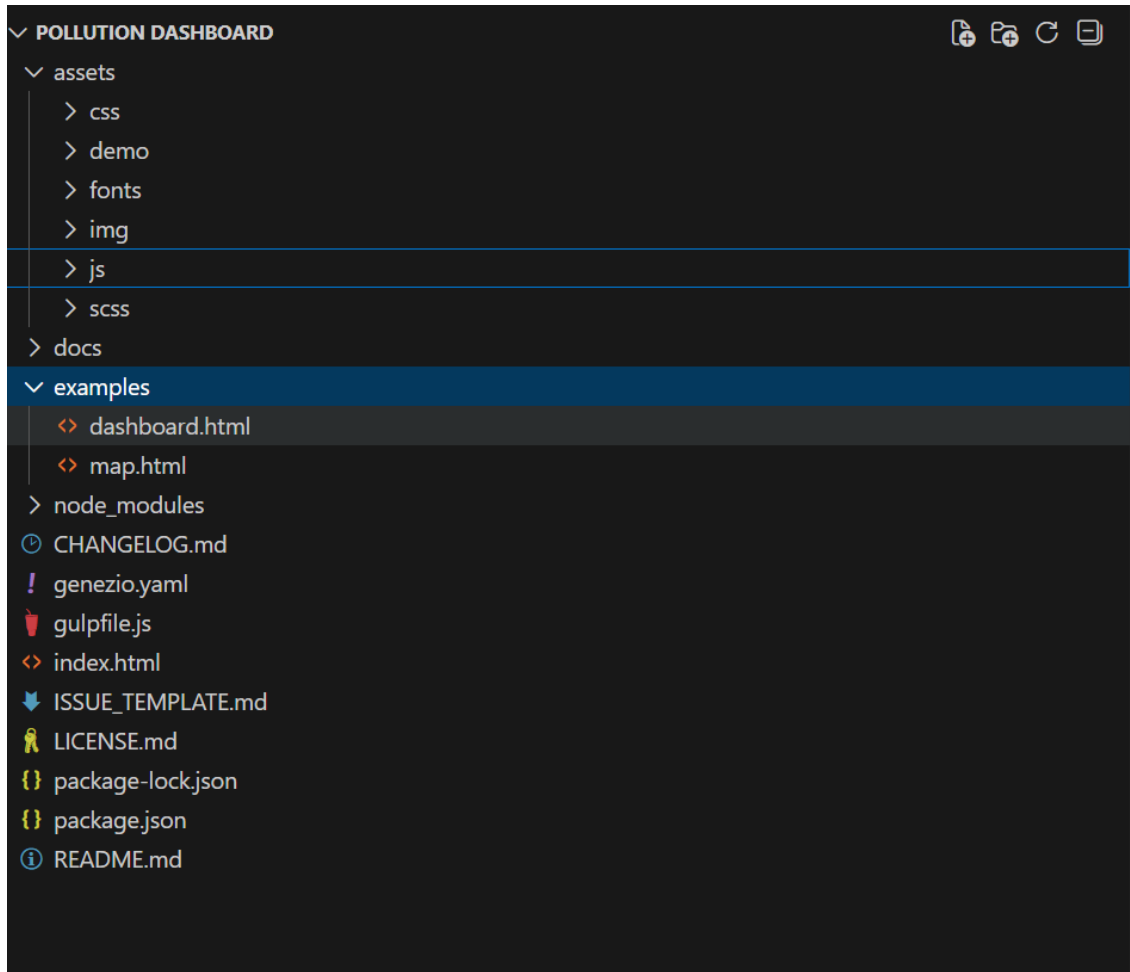


FIGURE 3.2 – Architecture générale de la solution (frontend statique + Azure Storage + APIs).

3.4 Mise en place du Storage Account

Un **Storage Account Azure** a été créé pour héberger l'application.

Les étapes principales sont :

1. création d'un groupe de ressources (*Resource Group*);
2. création d'un *Storage Account*;
3. activation de l'option *Static Website*;
4. déploiement des fichiers du projet dans le conteneur `$web`.

Une fois ces étapes terminées, Azure génère automatiquement une **URL publique**, permettant d'accéder à l'application depuis Internet.

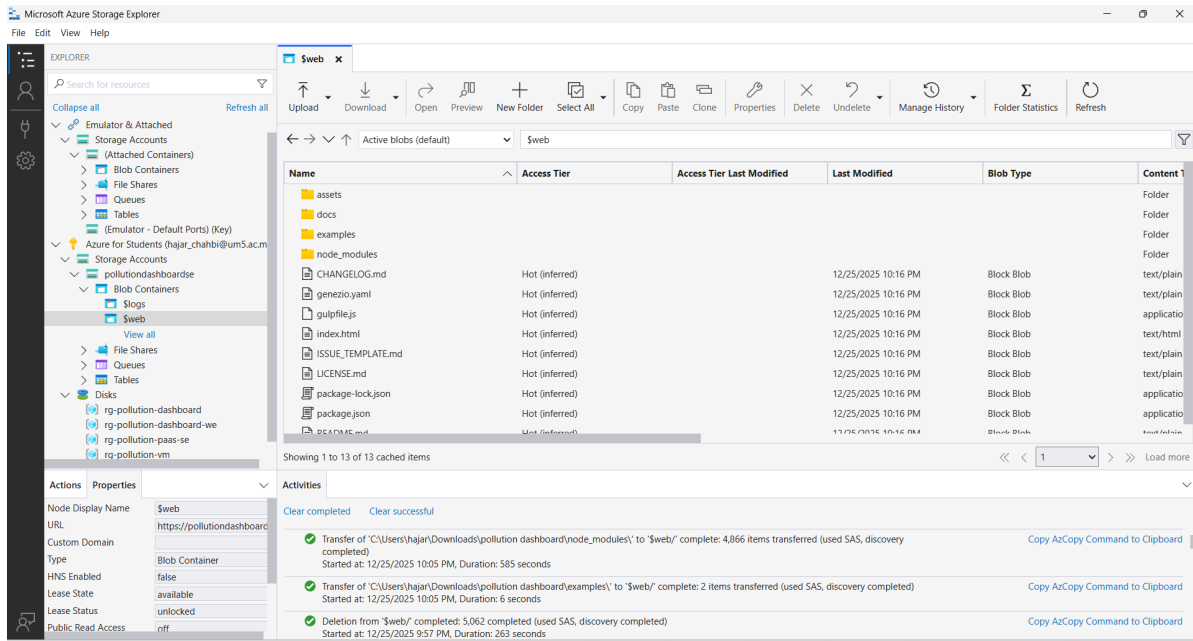


FIGURE 3.3 – Conteneur \$web dans Azure Storage Explorer pour l'hébergement du site statique.

3.5 Déploiement de l'application web

L'application a été déployée sous forme de site web statique comprenant :

- fichiers HTML (interface utilisateur),
- feuilles de style CSS,
- scripts JavaScript,
- fichiers de configuration.

Le déploiement a été réalisé via :

- Azure Storage Explorer,
- ou le portail Azure.

Cette méthode permet :

- une mise en ligne rapide,
- une grande stabilité,
- une compatibilité totale avec les navigateurs modernes.

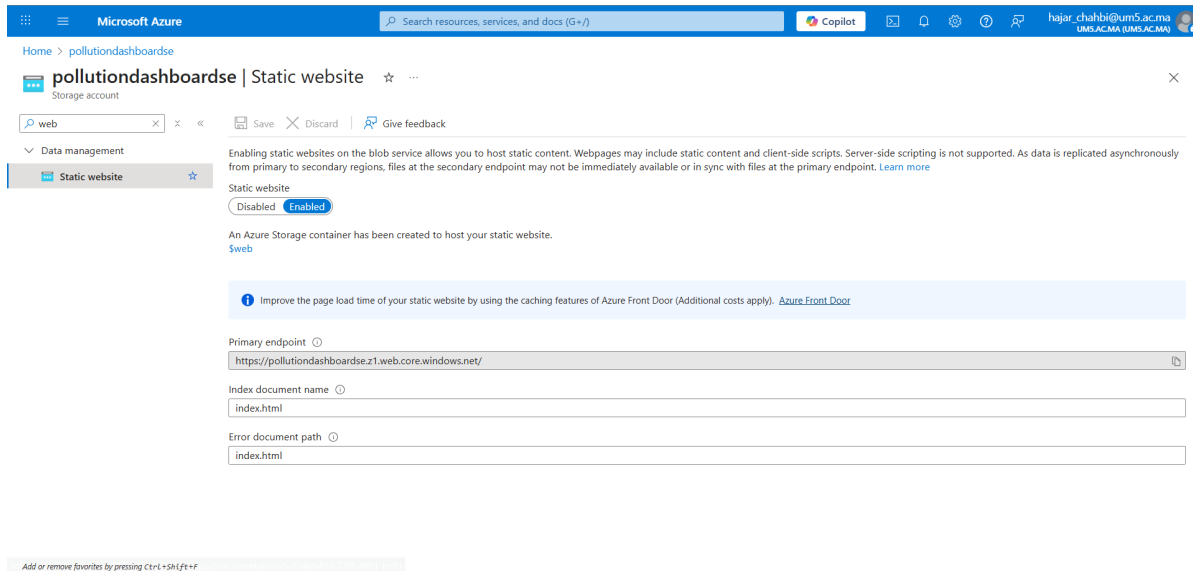


FIGURE 3.4 – Activation de l’hébergement statique et génération de l’URL publique (end-point).

3.6 Sécurité et accessibilité

L’application est accessible via une URL sécurisée fournie par Azure. Aucune authentification serveur n’est requise car l’application fonctionne uniquement côté client.

Les avantages de cette approche :

- pas de gestion de serveur ni de base de données locale,
- réduction des risques de panne,
- facilité de maintenance,
- coût réduit.

Dans un contexte académique, le stockage de la clé API côté client est acceptable, tout en restant vigilant (limitation des droits et usage raisonnable).

Chapitre 4

Présentation et fonctionnement de l'application Pollution Dashboard

4.1 Objectif de l'application

L'application **Pollution Dashboard** a été développée afin de visualiser et analyser la qualité de l'air dans plusieurs villes marocaines. Elle permet de consulter en temps réel l'indice de qualité de l'air (AQI), d'afficher des graphiques d'évolution et de localiser les villes sur une carte interactive.

L'objectif principal est de fournir une interface simple, accessible et dynamique permettant aux utilisateurs de suivre l'état de la pollution atmosphérique à partir de données réelles.

4.2 Accès à l'application

L'application est accessible en ligne via la plateforme Microsoft Azure à l'adresse suivante :

`https://pollutiondashboardse.z1.web.core.windows.net/`

Elle est hébergée sous forme d'application web statique grâce au service **Azure Static Web Apps**, ce qui garantit une haute disponibilité et un accès rapide sans configuration serveur.

4.3 Architecture générale de l'application

L'application repose sur une architecture légère côté client, composée des éléments suivants :

- **HTML / CSS** : structure et mise en page de l'interface,
- **JavaScript** : logique applicative et traitement des données,
- **API OpenWeather** : récupération des données de pollution,
- **Chart.js** : visualisation graphique,
- **Leaflet.js** : affichage cartographique,
- **Azure Static Web Apps** : hébergement cloud.

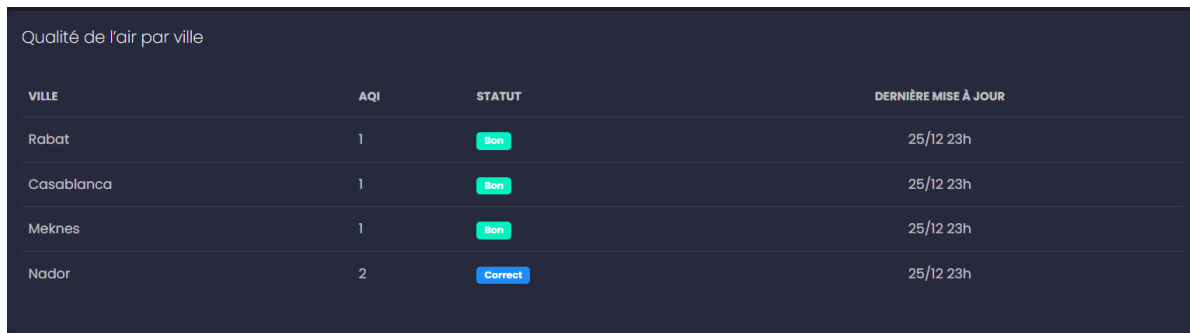
L'ensemble fonctionne sans backend, ce qui simplifie le déploiement et la maintenance.

4.4 Tableau de suivi de la qualité de l'air

L'application affiche un tableau récapitulatif contenant :

- le nom de la ville,
- la valeur de l'AQI,
- le statut de la qualité de l'air (Bon, Correct, Modéré, etc.),
- la date de la dernière mise à jour.

Les données sont récupérées dynamiquement depuis l'API OpenWeather et mises à jour automatiquement.



VILLE	AQI	STATUT	DERNIÈRE MISE À JOUR
Rabat	1	Bon	25/12 23h
Casablanca	1	Bon	25/12 23h
Meknes	1	Bon	25/12 23h
Nador	2	Correct	25/12 23h

FIGURE 4.1 – Tableau de suivi de la qualité de l'air

4.5 Visualisation graphique des données

L'application intègre plusieurs graphiques permettant d'analyser l'évolution de la pollution :

- courbes et graphiques des particules fines (PM2.5),
- comparaison entre plusieurs villes.

Ces graphiques sont générés dynamiquement à l'aide de la bibliothèque **Chart.js**.

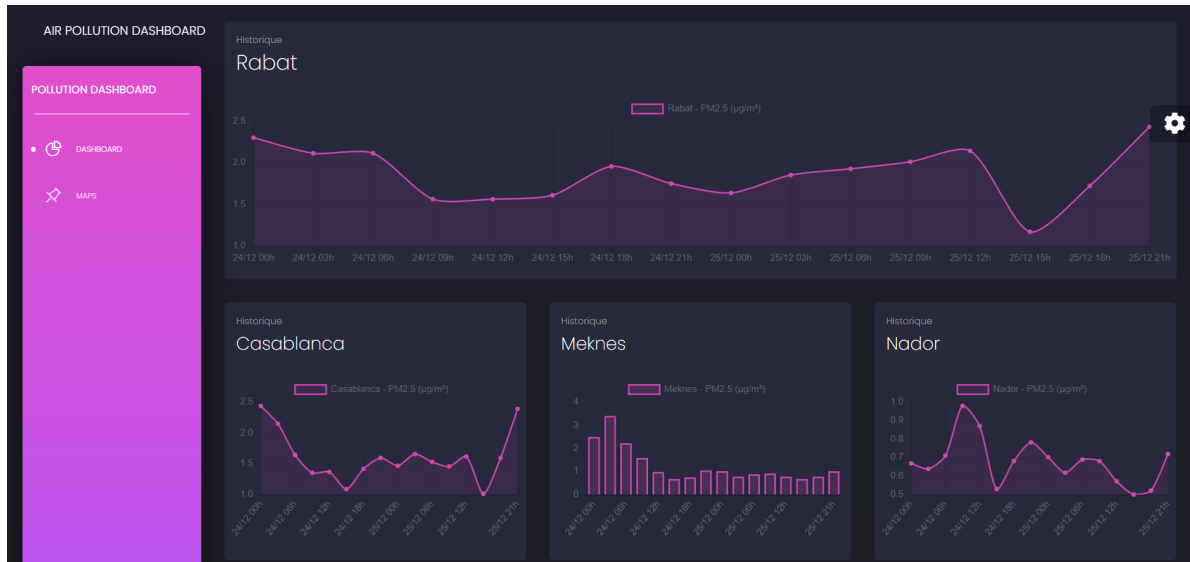


FIGURE 4.2 – Graphiques d'évolution de la qualité de l'air

4.6 Carte interactive

Une carte interactive a été intégrée afin de représenter géographiquement les données de pollution.

Elle permet :

- d'afficher la position des villes,
- de visualiser l'AQI directement sur la carte,
- d'interagir avec les marqueurs.

La carte repose sur la bibliothèque **Leaflet** et utilise les données de géolocalisation issues de l'API OpenWeather.

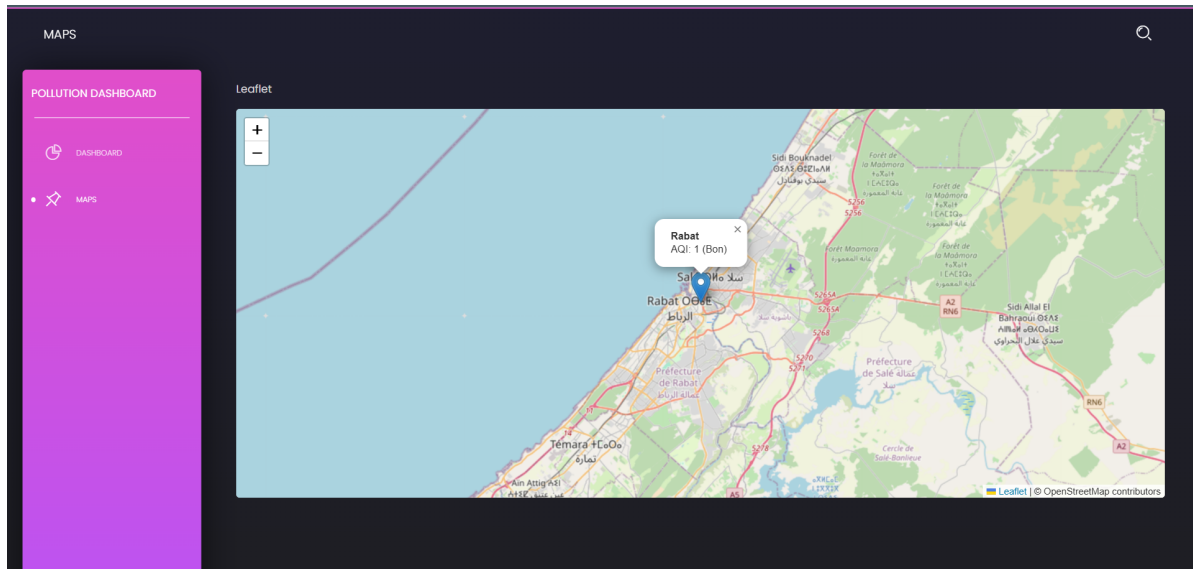


FIGURE 4.3 – Carte interactive de la qualité de l’air

4.7 Fonctionnalité de recherche

L’application dispose d’un champ de recherche permettant à l’utilisateur de saisir le nom d’une ville.

Cette fonctionnalité permet :

- de récupérer automatiquement les coordonnées géographiques,
- d’afficher la ville sur la carte,
- d’ajouter la ville dans le tableau,
- de mettre à jour les données de pollution.

Les villes recherchées sont enregistrées dans le navigateur via `localStorage`, ce qui permet de conserver l’historique même après rechargement de la page.

4.8 Synthèse du chapitre

Ce chapitre a permis de présenter :

- l’architecture générale de l’application,
- les fonctionnalités principales,
- la visualisation des données de pollution,
- l’intégration cartographique,

— l'accès public via Azure.

4.9 Limites fonctionnelles de l'application

Les chapitres précédents ont permis de présenter une première application de visualisation de la qualité de l'air, développée sous forme d'une application web statique et déployée sur Microsoft Azure. Cette application offre une consultation en temps réel des données environnementales à travers des graphiques, une carte interactive et un tableau récapitulatif.

Cependant, au cours de la réalisation et des tests de cette première version, certaines limites ont été identifiées. En effet, les données manipulées sont uniquement stockées localement dans le navigateur (*localStorage*), ce qui implique qu'elles sont perdues lors d'un rafraîchissement ou d'un changement d'appareil. De plus, l'application ne permet pas la gestion avancée des utilisateurs, ni l'envoi de notifications ou d'alertes automatiques en cas de dégradation de la qualité de l'air.

Face à ces limitations, il est apparu nécessaire de concevoir une seconde application plus complète, intégrant un véritable backend, une base de données persistante et des services automatisés. C'est dans cette optique qu'a été développée l'application *Pollution Predictor*, basée sur la plateforme Supabase, qui permet la gestion des données, l'authentification des utilisateurs ainsi que l'envoi d'alertes par email.

Le chapitre suivant présente en détail l'architecture, le fonctionnement et les composants de cette seconde application, conçue comme une évolution logique de la première.

Chapitre 5

Présentation de la plateforme Supabase

5.1 Supabase

Supabase est une plateforme open-source qui fournit une base de données PostgreSQL hébergée dans le cloud, accompagnée de services d'authentification et d'API automatiques.

Dans ce projet, Supabase est utilisé pour :

- le stockage des données de pollution ;
- la gestion des tables et des requêtes ;
- la sécurisation des accès aux données ;
- la communication entre l'application et la base de données.

L'utilisation de Supabase permet de simplifier le développement tout en garantissant de bonnes performances et une structure de données fiable.

5.1.1 Architecture de Supabase

L'architecture de Supabase repose sur plusieurs composants open source bien établis :

- **PostgreSQL** : moteur de base de données relationnelle,
- **PostgREST** : génération automatique d'API REST à partir des tables,
- **GoTrue** : gestion de l'authentification et des utilisateurs,
- **Storage API** : gestion du stockage de fichiers,
- **Edge Functions** : exécution de logique backend serverless.

Cette architecture permet une forte scalabilité, une séparation claire des responsabilités et une intégration directe avec les applications frontend modernes.

5.1.2 Base de données et création des tables

Supabase fournit une interface graphique facilitant la création et la gestion des tables PostgreSQL .

La Figure 5.1 illustre le processus de création d’une table dans le schéma `public`.

Create a new table under `public`

Name

Description

☒ Enable Row Level Security (RLS) RECOMMENDED
Restrict access to your table by enabling RLS and writing Postgres policies.

Policies are required to query data
You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an **empty array** of results. You can create policies after saving this table.
[Documentation](#)

☐ Enable Realtime
Broadcast changes on this table to authorized subscribers

Columns [About data types](#) [Import data from CSV](#)

Name	Type	Default Value	Primary
id	int8	NULL	☒
created_at	timestamp	now()	☐

[Add column](#)

[Cancel](#) [Save](#)

FIGURE 5.1 – Création d’une table dans Supabase

Chaque table peut contenir :

- une clé primaire,
- des types de données PostgreSQL standards,
- des valeurs par défaut,
- des règles de sécurité associées.

Row Level Security (RLS)

Le **Row Level Security (RLS)** est une fonctionnalité native de PostgreSQL permettant de restreindre l'accès aux lignes d'une table en fonction de politiques définies.

Lorsque le RLS est désactivé, comme montré dans la Figure 5.2, toutes les requêtes autorisées peuvent accéder aux données sans filtrage.



FIGURE 5.2 – RLS désactivé dans Supabase

Cette configuration est acceptable en phase de développement, mais elle doit être activée en environnement de production afin de garantir la sécurité des données.

5.1.3 API automatique

À partir des tables PostgreSQL, Supabase génère automatiquement :

- une API REST conforme à PostgREST,
- une API GraphQL.

Cette génération automatique permet d'effectuer des opérations CRUD (Create, Read, Update, Delete) sans implémenter de code backend supplémentaire.

5.1.4 Edge Functions

Les **Edge Functions** sont des fonctions serverless exécutées dans un environnement Deno, déployées globalement afin de réduire la latence.

La Figure 5.3 présente l'interface de gestion des Edge Functions.

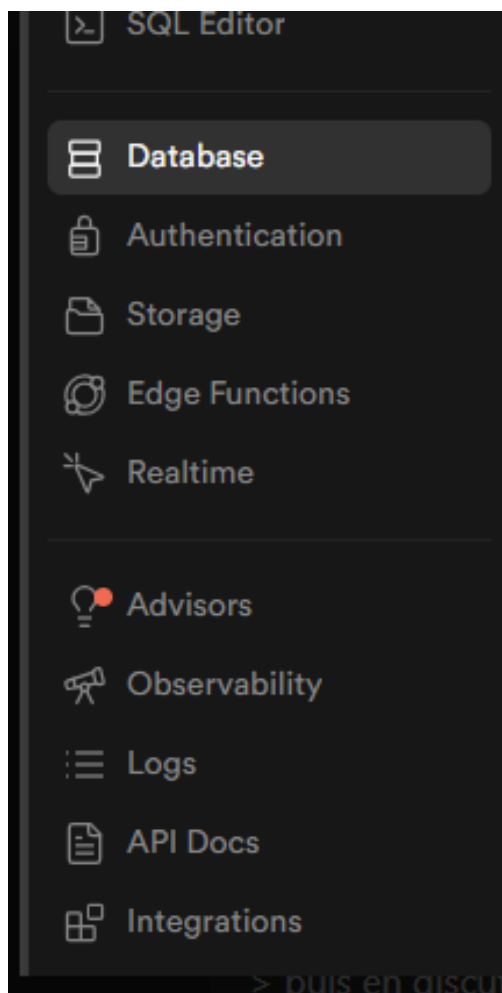


FIGURE 5.3 – Gestion des Edge Functions dans Supabase

Overview

La section *Overview* fournit une vue globale des performances de la fonction, incluant le temps moyen d'exécution et le nombre total d'appels.

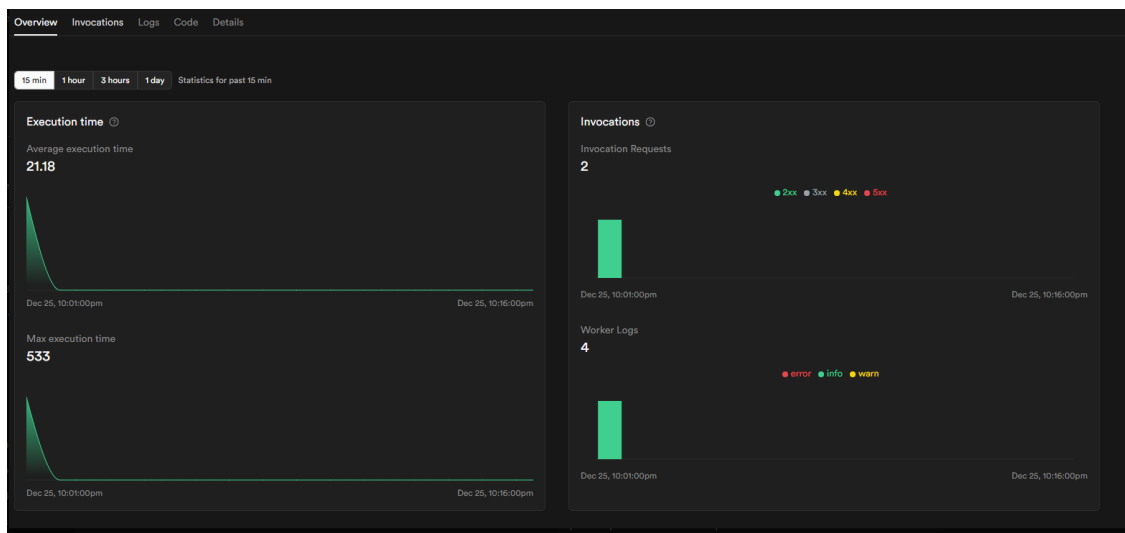


FIGURE 5.4 – Vue d’ensemble d’une Edge Function

Invocations

La section *Invocations* affiche l’historique détaillé des appels, avec les codes de réponse HTTP et les horodatages.

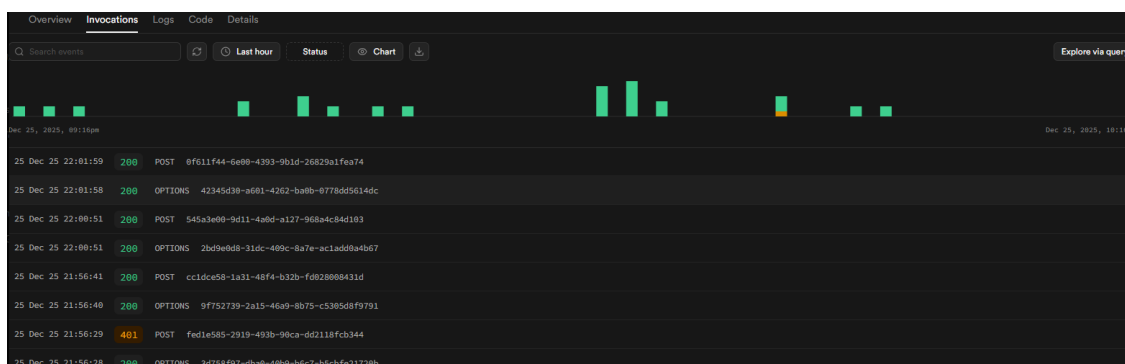


FIGURE 5.5 – Historique des invocations d’une Edge Function

Logs

Les logs permettent d’analyser le comportement de la fonction et de détecter d’éventuelles erreurs d’exécution.

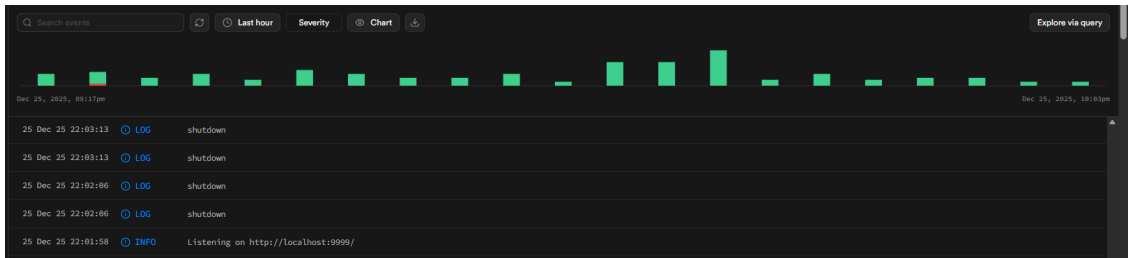


FIGURE 5.6 – Logs d'exécution d'une Edge Function

Code et déploiement

Les Edge Functions permettent de déployer du code backend sans gestion d'infrastructure serveur.

```

1 import { serve } from "https://deno.land/std@0.177.0/http/server.ts";
2 import { createClient } from "https://esm.sh/@supabase/supabase-js@2";
3
4 const corsHeaders = {
5   "Access-Control-Allow-Origin": "*",
6   "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type",
7   "Access-Control-Allow-Methods": "POST, OPTIONS",
8 };
9
10 serve(async (req) => {
11   if (req.method === "OPTIONS") {
12     return new Response("ok", { status: 200, headers: corsHeaders });
13   }
14
15   try {
16     const body = await req.json();
17     const cityInput = body.city;
18     const latInput = body.lat;
19     const lonInput = body.lon;
20
21     const supabase = createClient(
22       Deno.env.get("SUPABASE_URL")!,
23       Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
24     );
25
26     const apikey = Deno.env.get("OPENWEATHER_API_KEY");
27
28     let lat: number, lon: number, cityResolved = cityInput;
29
30     // Si lat/lon envoyés depuis le frontend
31     if (latInput && lonInput) {
32       lat = latInput;
33       lon = lonInput;
34
35       // Optionnel : récupérer le nom de la ville via reverse geocoding
36       try {

```

FIGURE 5.7 – Code source d'une Edge Function

Le déploiement permet d'intégrer des API externes, d'exécuter des traitements sécurisés et de centraliser la logique backend.

Détails et vérification JWT des Edge Functions

La section *Details* d'une Edge Function Supabase regroupe les informations principales liées à l'identification, au déploiement et à la sécurité de la fonction. Elle permet notamment d'accéder au *slug*, à l'URL d'invocation ainsi qu'aux paramètres de configuration de la fonction.

La Figure 5.8 illustre la section *Details* de la fonction `collect-pollution` utilisée dans ce projet.

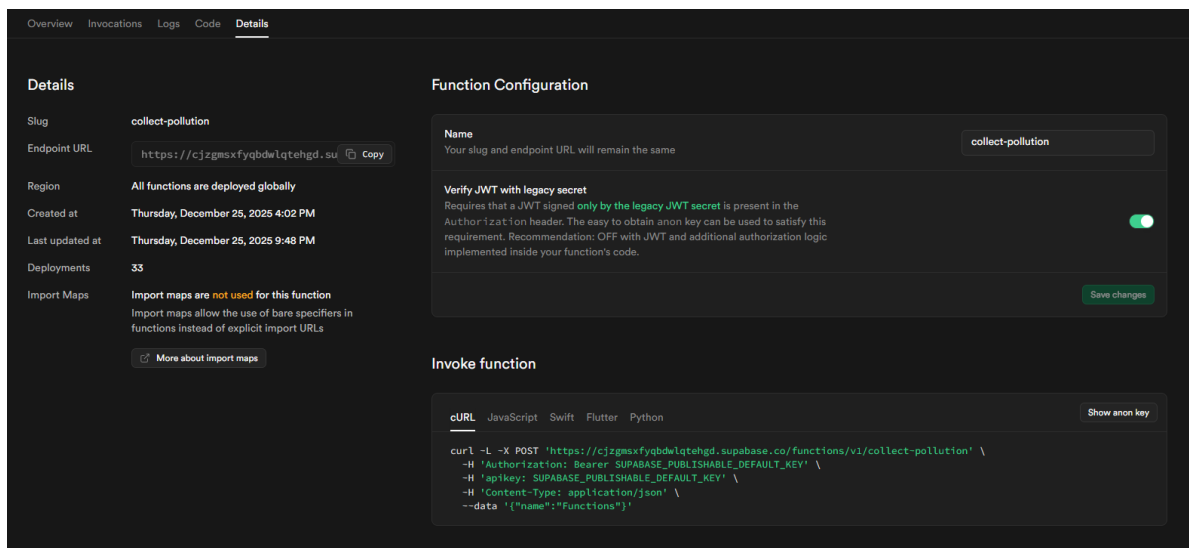


FIGURE 5.8 – Section *Details* d’une Edge Function Supabase

L’option *Verify JWT with legacy secret* active la vérification automatique des requêtes entrantes à l’aide d’un **JSON Web Token (JWT)** signé avec le secret legacy du projet. Cette vérification garantit que seules des requêtes autorisées peuvent invoquer la fonction, renforçant ainsi la sécurité du backend.

Dans le cadre de ce projet, cette configuration permet de sécuriser l’accès à la fonction `collect-pollution`, responsable de la collecte et du stockage des données de qualité de l’air dans la base de données Supabase.

5.1.5 Authentification et sécurité

Supabase intègre un système d’authentification basé sur JWT, permettant la gestion des utilisateurs, des sessions et des permissions, tout en respectant les bonnes pratiques de sécurité.

5.2 Justification du choix des technologies

Le choix des technologies utilisées dans ce projet repose sur plusieurs critères :

- la facilité d’intégration ;
- la disponibilité des services cloud ;
- la fiabilité et la sécurité des données ;
- la compatibilité avec les applications web modernes ;
- la gratuité ou l’existence d’une offre académique.

Microsoft Azure et Supabase ont été retenus car ils répondent efficacement à ces critères tout en offrant une grande flexibilité pour le développement d'applications cloud.

Chapitre 6

Fonctionnement global de l'application Pollution Predictor avec supabase et alerte par Gmail

Ce chapitre présente le fonctionnement général de l'application *Pollution Predictor*. L'objectif principal de cette application est de permettre aux utilisateurs de consulter la qualité de l'air au Maroc, de s'inscrire aux alertes de pollution et de communiquer avec l'administration du système.

L'application est organisée autour de trois grandes parties :

- un **dashboard de visualisation de la pollution**,
- un **module d'inscription aux alertes de pollution**,
- un **module de contact et de désinscription**.

6.1 Présentation générale de l'application

Le site web *Pollution Predictor* est une application web interactive composée de trois interfaces principales :

- **Dashboard** : permet à l'utilisateur de consulter la qualité de l'air d'une ville donnée au Maroc ;
- **Inscription aux alertes** : permet aux utilisateurs de s'inscrire pour recevoir des alertes de pollution par email ;
- **Contact / Désinscription** : permet d'envoyer un message à l'administration ou de se désabonner des alertes.

Cette organisation assure une navigation simple et une séparation claire des fonctionnalités.

6.2 Fonctionnement du dashboard de qualité de l'air

Le dashboard constitue la partie centrale de l'application. Il permet à l'utilisateur d'accéder aux informations de pollution de manière interactive.

6.2.1 Interaction utilisateur et récupération des données

L'utilisateur peut consulter la pollution de deux manières :

- en saisissant le nom d'une ville dans le champ de recherche ;
- en effectuant un double-clic sur la carte interactive.

Dans les deux cas, les coordonnées géographiques (latitude et longitude) sont utilisées pour récupérer les données de pollution depuis l'API *OpenWeather Air Pollution*.

La Figure 6.1 illustre l'interface du dashboard avec la carte et les informations de pollution affichées.

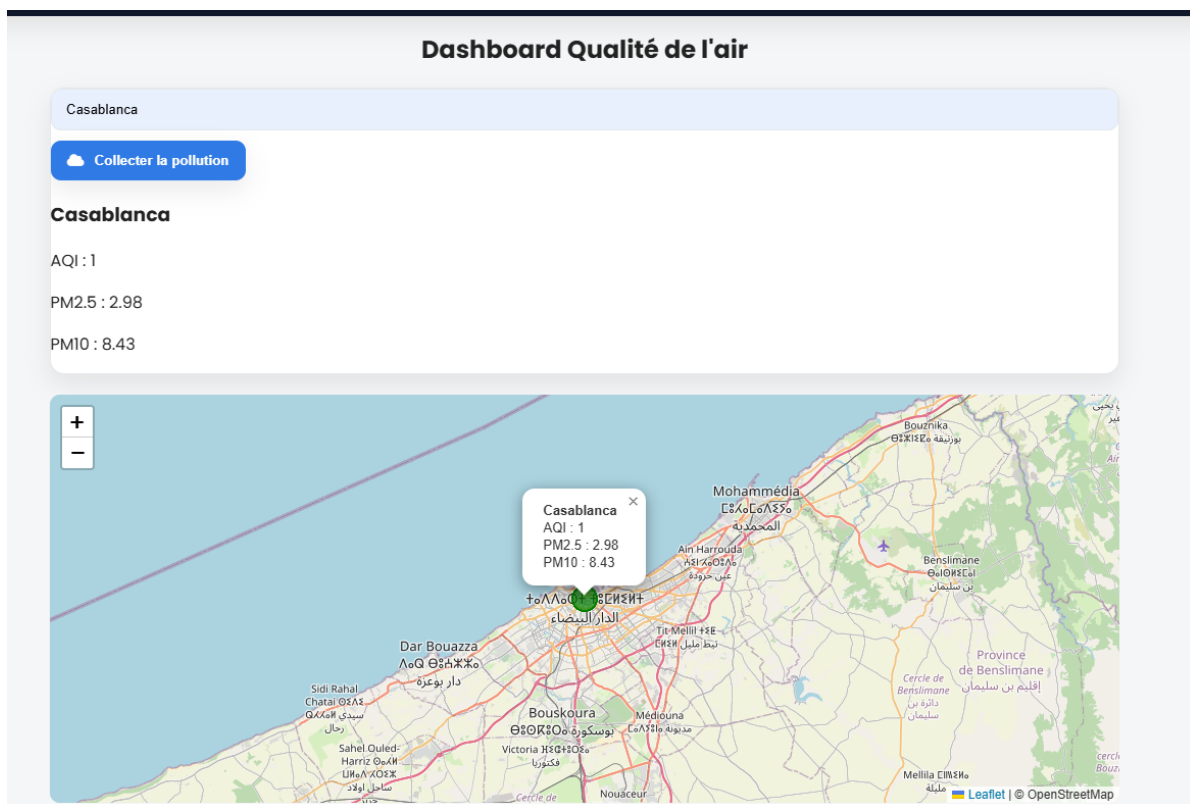


FIGURE 6.1 – Dashboard de qualité de l'air avec carte interactive

6.2.2 Indicateurs de pollution affichés

Les indicateurs affichés sont :

- **AQI (Air Quality Index)** : indice global de la qualité de l'air (plus il est élevé, plus l'air est pollué) ;
- **PM2.5** : concentration de particules fines inférieures à 2.5 micromètres ;
- **PM10** : concentration de particules inférieures à 10 micromètres.

Ces indicateurs permettent d'évaluer les risques pour la santé humaine.

6.2.3 Historique et visualisation graphique

Les données récupérées sont stockées dans la base de données Supabase, puis exploitées pour afficher un historique sous forme de graphique.

La Figure 6.2 présente l'évolution de l'AQI et du PM2.5 dans le temps pour une ville donnée.

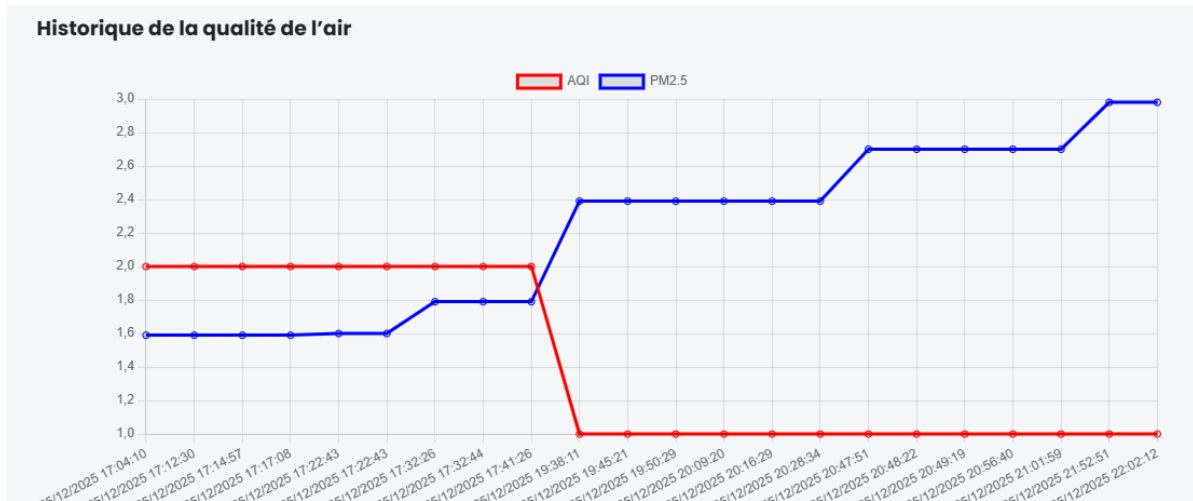


FIGURE 6.2 – Historique de la qualité de l'air (AQI et PM2.5)

6.3 Inscription aux alertes de pollution

Cette partie permet aux utilisateurs de recevoir automatiquement des alertes lorsque la pollution devient dangereuse dans leur ville.

6.3.1 Formulaire d'inscription

L'utilisateur renseigne son nom, son email et sa ville via un formulaire dédié, comme illustré dans la Figure 6.3.

Inscription aux alertes de pollution

Recevez des notifications pour votre ville au Maroc

Nom

Votre nom

Email

Votre email

Ville (Maroc)

Ex: rabat, casablanca, meknes...

S'inscrire

FIGURE 6.3 – Formulaire d’inscription aux alertes de pollution

6.3.2 Stockage des inscriptions

Les informations saisies sont envoyées au backend via une Edge Function (`signup`) puis stockées dans la base de données Supabase.

La Figure 6.4 montre la table contenant les utilisateurs inscrits.

	name	email	city	active	createdAt
5	Nirmine	nirmihanani2@gmail.com	Bouti Sayah	TRUE	2025-12-25 19:17:47.32

FIGURE 6.4 – Table des utilisateurs inscrits aux alertes

6.4 Module Contact et Désinscription

Le module de contact permet à l’utilisateur :

- d’envoyer un message à l’administration ;
- de se désabonner des alertes de pollution.

6.4.1 Formulaire de contact

La Figure 6.5 présente le formulaire de contact et de désinscription.

Contact / Désinscription

Écrivez-nous ou désinscrivez-vous des alertes

Nom

nirmine

Email

maisonifia@gmail.com

Action

Se désabonner

Message

Votre message (obligatoire si message)

Envoyer

FIGURE 6.5 – Formulaire de contact et de désinscription

6.4.2 Gestion côté backend

Selon l'action choisie, une Edge Function Supabase :

- enregistre le message dans la table **contact** ;
- ou supprime automatiquement l'utilisateur de la table **users** en cas de désinscription.

La Figure 6.6 illustre les données stockées dans la table de contact.

id	created_at	name	email	message	type
1	2025-12-25 18:37:43.748152+00	nirmine	maisonifia@gmail.com	lui	message
2	2025-12-25 18:54:55.874052+00	nirmine	maisonifia@gmail.com	NULL	unsubscribe
3	2025-12-25 19:13:49.962626+00	nirmine	maisonifia@gmail.com	NULL	unsubscribe
4	2025-12-25 19:14:12.211938+00	nirmine	maisonifia@gmail.com	salut	message
5	2025-12-25 19:17:32.560011+00	nirmine	nirmihaniani2@gmail.com	NULL	unsubscribe
6	2025-12-25 19:45:57.776435+00	nirmine	maisonifia@gmail.com	NULL	unsubscribe

FIGURE 6.6 – Table des messages et désinscriptions

6.5 Système d'alertes email

6.5.1 Configuration du service email

Pour l'envoi des alertes, le service **EmailJS** est utilisé. Un service Gmail est configuré et relié à l'application.

La Figure 6.7 montre la configuration du service email.

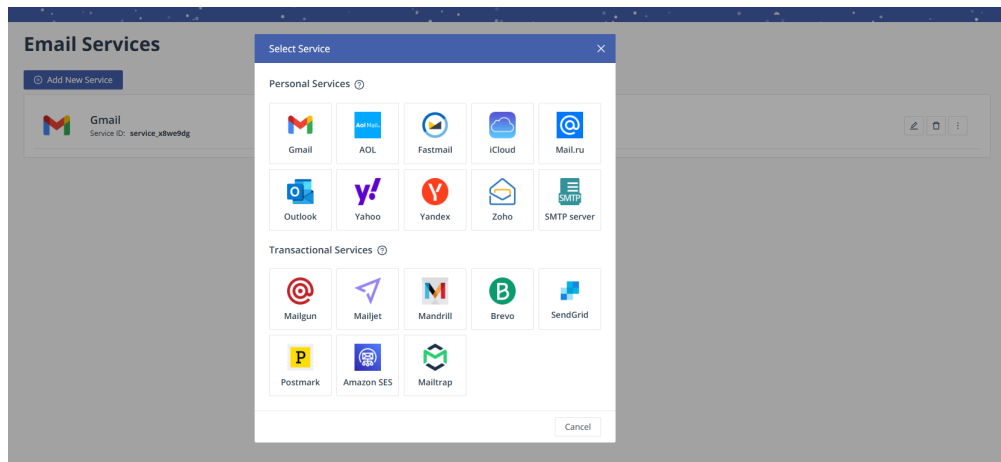


FIGURE 6.7 – Configuration du service email avec EmailJS

6.5.2 Template d'email

Un template d'email est créé afin de personnaliser le contenu des alertes (ville, AQI, recommandations).

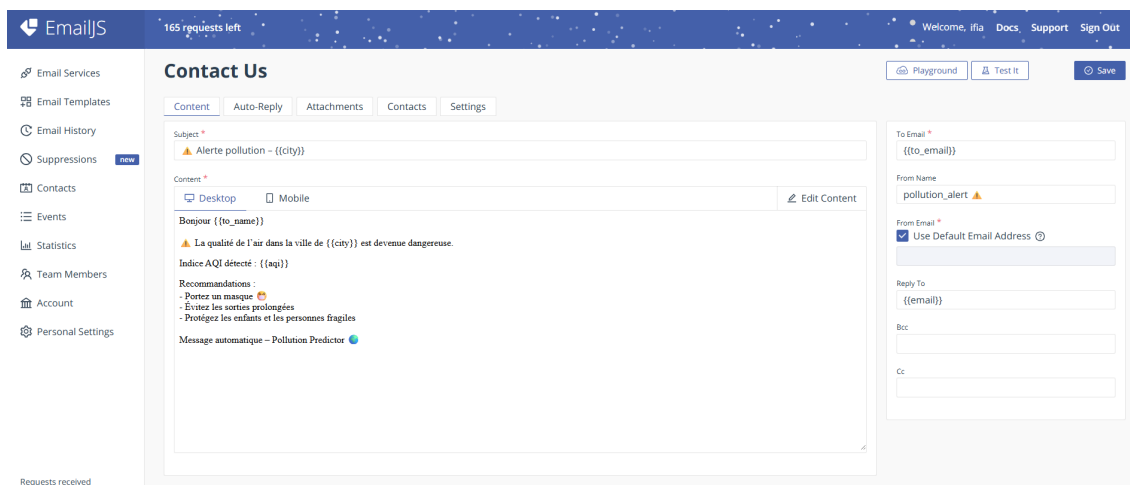


FIGURE 6.8 – Template d'email pour les alertes de pollution

6.5.3 Réception des alertes

Lorsque le seuil de pollution est dépassé, un email d'alerte est automatiquement envoyé à l'utilisateur inscrit.

La Figure 6.9 illustre un exemple d'email reçu par un utilisateur.



FIGURE 6.9 – Exemple d'alerte de pollution reçue par email

Ce mécanisme permet d'informer rapidement les utilisateurs et de renforcer la prévention face aux risques liés à la pollution atmosphérique.

Chapitre 7

Conclusion générale

Ce projet a permis de concevoir et de déployer une solution complète de surveillance et d'analyse de la qualité de l'air, reposant sur deux applications complémentaires : un tableau de bord de visualisation hébergé sur Microsoft Azure et une application avancée intégrant un backend basé sur Supabase.

La première application, déployée sous forme de site web statique, illustre l'utilisation efficace du cloud computing à travers le modèle PaaS. Elle permet la consultation en temps réel des données de pollution, leur visualisation graphique et leur représentation cartographique, tout en assurant une grande simplicité d'accès et de maintenance.

La seconde application, Pollution Predictor, enrichit cette approche par l'intégration de services backend avancés, incluant la gestion des données, l'automatisation des alertes et la communication par email. L'utilisation de Supabase et d'EmailJS a permis de mettre en œuvre une architecture moderne, sécurisée et évolutive.

Ce projet met en évidence l'intérêt du cloud computing pour le développement d'applications environnementales, en combinant accessibilité, performance et scalabilité. Il constitue également une base solide pour des évolutions futures, notamment l'intégration de modèles prédictifs basés sur le machine learning, l'automatisation avancée des alertes et l'exploitation de données environnementales plus fines.

En conclusion, ce travail a permis de consolider les compétences techniques acquises durant la formation, tout en illustrant l'apport des technologies cloud dans la résolution de problématiques réelles liées à la qualité de l'air.

Bibliographie

- [1] Microsoft Azure. *Documentation officielle Microsoft Azure*. Disponible sur : <https://learn.microsoft.com/fr-fr/azure/>.
- [2] Supabase. *Documentation officielle Supabase*. Disponible sur : <https://supabase.com/docs>.
- [3] OpenWeather API. *API de données météorologiques et de pollution*. Disponible sur : <https://openweathermap.org/api>.
- [4] Mell, P., Grance, T. *The NIST Definition of Cloud Computing*. NIST Special Publication 800-145, 2011.
- [5] Mozilla Developer Network (MDN). *Documentation JavaScript et développement Web*. Disponible sur : <https://developer.mozilla.org/>.
- [6] World Air Quality Index Project. *Air Quality Index (AQI) Basics*. Disponible sur : <https://aqicn.org/scale/>.
- [7] EmailJS. *Documentation officielle EmailJS*. Disponible sur : <https://www.emailjs.com/docs/>.
- [8] Supabase. *Documentation officielle Supabase*. Disponible en ligne : <https://supabase.com/docs>.